BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO Lab02 – Giải quyết vấn đề bằng tìm kiếm

MÔN: TRÍ TUỆ NHÂN TẠO NÂNG CAO

Giảng viên hướng dẫn: Đỗ Như Tài

Sinh viên thực hiện:
Đỗ Xuân Triệu - 3124720072
Huỳnh Tấn Phi Hùng - 3124720026
Lê Tấn Phát – 3124720050

TP.HỒ CHÍ MINH, ngày 10 tháng 9 năm 2026

Contents

MỤC TIÊU VÀ MÔ HÌNH BÀI TOÁN 1

a. Các thành phần của bài toán tìm kiếm 1

b. Các chỉ số dùng trong báo cáo 1

A. BÀI TẬP THỰC HÀNH 3

BÀI 1. CẤU TRÚC CÂY TÌM KIẾM VÀ LỚP NODE 3

BÀI 2. MÔ HÌNH HÓA MÊ CUNG 3

2.1. Cây tìm kiếm và đồ thị trạng thái 4

BÀI 3. TÌM KIẾM KHÔNG THÔNG TIN: BFS VÀ DFS 4

3.1. Nguyên lý 4

3.2. Kết quả BFS và DFS 5

3.3. Phân tích 6

3.4. Tính đầy đủ, tối ưu và bộ nhớ 7

BÀI 4. DLS VÀ IDS 7

4.1. Depth-Limited Search (DLS) 7

4.2. Iterative Deepening Search (IDS) 7

BÀI 5. TÌM KIẾM CÓ THÔNG TIN: GBFS, A* VÀ WEIGHTED A* 8

5.1. Greedy Best-First Search (GBFS) 8

5.2. Thuật toán A* 9

5.3. Thuật toán Weighted A* 11

BÀI 6. ẢNH HƯỞNG CỦA HEURISTIC 11

6.1. Quan sát bản đồ giá trị $\boldsymbol{hn}$ và $\boldsymbol{fn}$ trong mê cung rỗng 11

6.2. So sánh Manhattan và Euclidean cho $\boldsymbol{A*}$ trong lưới 4 hướng 12

6.3. Đề xuất Heuristic mới và Điều kiện đảm bảo tính tối ưu 13

BÀI 7. CYCLE CHECKING 13

7.1. Thứ tự sinh trạng thái và chi phí quản lý 14

BÀI 8. GOAL-BASED PLANNING AGENT VÀ NHIỀU MỤC TIÊU 14

TỔNG HỢP KẾT QUẢ THỰC NGHIỆM 15

BẢNG KẾT QUẢ SO SÁNH CÁC THUẬT TOÁN TÌM KIẾM 17

Nhận xét từ bảng kết quả 18

NHẬN XÉT VÀ ĐÁNH GIÁ 19

B. CÂU HỎI THẢO LUẬN 21

Câu 6: Khi nào BFS dùng nhiều bộ nhớ hơn A*? Vì sao heuristic tốt làm giảm vùng trạng thái phải khảo sát? 21

Câu 7. Vì sao DFS có thể cho đường đi dài hoặc gặp vấn đề trong môi trường có chu trình? 21
Câu 8. Vì sao thứ tự sinh trạng thái con ảnh hưởng mạnh đến DFS nhưng ít ảnh hưởng hơn đến BFS? 22
Câu 9: So sánh GBFS và A*: thuật toán nào sử dụng $\boldsymbol{gn}$, $\boldsymbol{hn}$, $\boldsymbol{fn}$? Ảnh hưởng đến tính tối ưu như thế nào? 22
Câu 10: Manhattan và Euclidean khác nhau ra sao trong lưới chỉ cho phép đi lên/xuống/trái/phải? ... 22
Câu 11: Weighted A* thay đổi điều gì khi tăng trọng số heuristic?..... 23
Câu 12. Cycle checking giảm lặp trạng thái nhưng làm tăng chi phí quản lý dữ liệu như thế nào? 23
C. KẾT LUẬN 23
BẢNG PHÂN CÔNG: 24

MỤC TIÊU VÀ MÔ HÌNH BÀI TOÁN

Lab 02 nghiên cứu cách biểu diễn bài toán tìm kiếm bằng không gian trạng thái và đánh giá các chiến lược tìm kiếm trên mê cung. Các thuật toán được khảo sát gồm BFS, DFS, Depth-Limited Search (DLS), Iterative Deepening Search (IDS), Greedy Best-First Search (GBFS), A* và Weighted A*. Việc đánh giá dựa trên chi phí đường đi, số node mở rộng, độ sâu, bộ nhớ, frontier và thời gian chạy khi các chỉ số có trong output.

a. Các thành phần của bài toán tìm kiếm

Thành phần	Ý nghĩa trong mê cung
Không gian trạng thái	Các vị trí hợp lệ mà tác tử có thể đến.
Trạng thái đầu	Vị trí xuất phát S.
Trạng thái đích	Vị trí cần đến G.
Hành động	Di chuyển lên, xuống, trái hoặc phải nếu ô kế tiếp hợp lệ.
Hàm chuyển trạng thái	Xác định vị trí mới sau một hành động.
Chi phí đường đi	Tổng chi phí các bước; trong các thí nghiệm này, mỗi bước hợp lệ có chi phí 1.

```python
class Node:
    def __init__(self, pos, parent, action, cost):
        self.pos = tuple(pos)    # the state; positions are (row,col)
        self.parent = parent     # reference to parent node. None means root node.
        self.action = action     # action used in the transition function (root node has None)
        self.cost = cost         # for uniform cost this is the depth. It is also g(n) for A* search

    def get_path_from_root(self):
        """returns nodes on the path from the root to the current node."""
        node = self
        path = [node]

        while not node.parent is None:
            node = node.parent
            path.append(node)

        path.reverse()

        return(path)
```

Vì mỗi bước có chi phí bằng nhau, path cost tương ứng với số bước di chuyển. Do đó, BFS là mốc đối chiếu phù hợp để kiểm tra đường đi tối ưu trong các mê cung được khảo sát.

b. Các chỉ số dùng trong báo cáo

Chỉ số	Cách hiểu
Path cost / path length	Tổng chi phí hoặc số bước của đường đi tìm được.
Expanded	Số node được mở rộng trong quá trình tìm kiếm.

Depth	Độ sâu của node hoặc lời giải theo cách ghi của output.
Max frontier	Số node chờ xử lý lớn nhất tại một thời điểm.
Max memory	Chỉ số bộ nhớ/tập node tối đa được báo cáo bởi chương trình; cần phân biệt với max frontier.
Wall time	Thời gian thực thi; các giá trị nhỏ có thể đã được làm tròn khi hiển thị.

A. BÀI TẬP THỰC HÀNH

BÀI 1. CẤU TRÚC CÂY TÌM KIẾM VÀ LỚP NODE

Một node trong cây tìm kiếm biểu diễn một trạng thái cùng thông tin cần thiết để truy vết lời giải. Trong bài thực hành, node gồm các thuộc tính chính:

Thuộc tính	Vai trò
pos	Tọa độ của trạng thái hiện tại.
parent	Node cha để truy vết về trạng thái đầu.
action	Hành động đã đưa từ node cha đến node hiện tại.
cost	Chi phí tích lũy từ node gốc đến node hiện tại.

Ví dụ output minh họa Positions = [(0, 0), (0, 1)], Actions = [None, 'E'] và Path cost = [0, 1]. Trạng thái đầu có chi phí 0; sau một bước sang Đông (E), chi phí tăng lên 1. Khi tìm được đích, chương trình lần ngược qua parent để dựng đường đi rồi đảo thứ tự để thu được chuỗi từ trạng thái đầu đến đích.

Cần phân biệt node với trạng thái: cùng một tọa độ có thể được biểu diễn bởi nhiều node nếu được phát hiện từ những đường đi khác nhau. Cơ chế kiểm tra trạng thái trùng giúp giảm công việc lặp và có thể ảnh hưởng đến tính đúng đắn, thời gian và bộ nhớ của thuật toán.

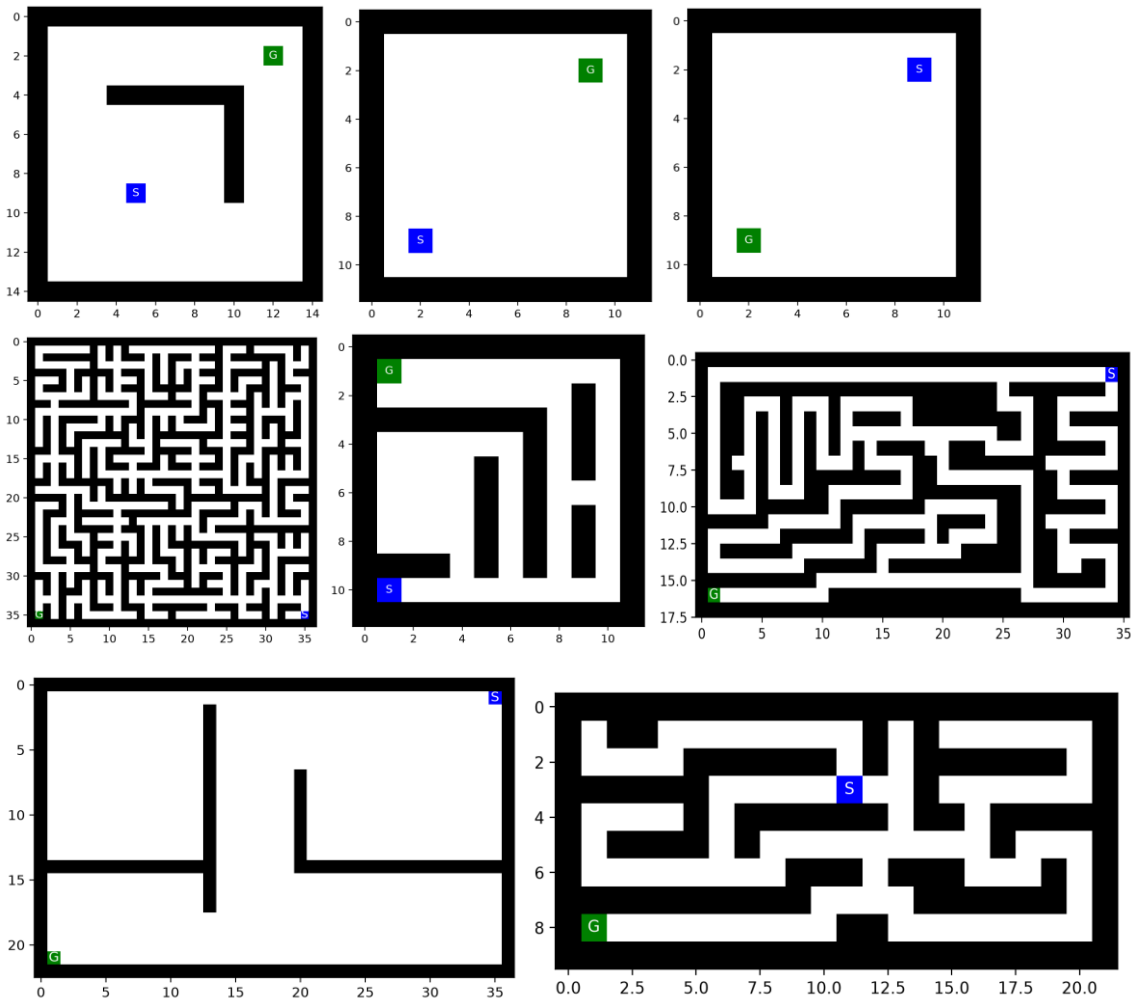
BÀI 2. MÔ HÌNH HÓA MÊ CUNG

Trong mô hình mê cung, ký tự X biểu diễn tường; các ô không phải X được xem là ô có thể đi qua theo quy ước của tệp dữ liệu. Từ mỗi ô, tác tử có tối đa bốn hướng đi, do đó hệ số phân nhánh b không vượt quá 4.

Bảng dưới đây tổng hợp kích thước tĩnh n (số ký tự không phải X được đếm trong tệp mê cung) và các chỉ số BFS đã lưu. n không phải số node mà BFS mở rộng; d là path cost BFS và m là độ sâu BFS quan sát được trong output.

Mê cung	n	d (BFS cost)	m (BFS depth quan sát)	b tối đa
small	94	19	19	≤ 4
medium	274	68	68	≤ 4
large	647	210	210	≤ 4
open	684	54	54	≤ 4
L	157	16	16	≤ 4
loops	72	23	23	≤ 4
empty	100	14	14	≤ 4
empty_2	100	14	14	≤ 4

Kết quả cho thấy độ dài đường đi khác nhau đáng kể giữa các mê cung. Tuy nhiên, kích thước mê cung n không quyết định trực tiếp path cost: hình dạng tường và vị trí tương đối của điểm đầu, điểm đích cũng ảnh hưởng đến đường đi.



2.1. Cây tìm kiếm và đồ thị trạng thái

Cây tìm kiếm biểu diễn các đường đi được sinh ra từ trạng thái đầu, trong khi đồ thị trạng thái biểu diễn các trạng thái và các chuyển tiếp có thể có. Trong mê cung có chu trình, nhiều chuỗi hành động khác nhau có thể dẫn về cùng một tọa độ. Nếu không kiểm soát trạng thái trùng, cây tìm kiếm có thể tiếp tục sinh các nhánh lặp dù số trạng thái thực tế là hữu hạn.

BÀI 3. TÌM KIẾM KHÔNG THÔNG TIN: BFS VÀ DFS

3.1. Nguyên lý

BFS (Breadth-First Search) dùng hàng đợi FIFO và mở rộng các node theo từng mức độ sâu. Khi mỗi bước có chi phí bằng nhau, BFS tìm được đường đi ít bước nhất nếu có lời giải và trạng thái trùng được xử lý đúng.

DFS (Depth-First Search) dùng ngăn xếp LIFO hoặc đệ quy để đi sâu theo một nhánh trước khi quay lui. DFS có thể giữ frontier nhỏ hơn trong nhiều trường hợp, nhưng đường đi phụ thuộc thứ tự sinh trạng thái con và không bảo đảm tối ưu.

```python
def bfs(maze, start, goal, node_limit=float("inf"), time_limit=float("inf")):
    """BFS: FIFO frontier, `reached` prevents repeated states."""
    metrics = new_metrics()
    start, goal = tuple(int(v) for v in start), tuple(int(v) for v in goal)
    t0 = time.perf_counter()

    root = Node(start, None, None, 0)
    if start == goal:
        return root, metrics

    frontier = deque([root])
    reached = {start}

    while frontier:
        if metrics["expanded"] >= node_limit or time.perf_counter() - t0 > time_limit:
            metrics["limit"] = "node/time budget"
            return None, metrics

        node = frontier.popleft()
        metrics["expanded"] += 1

        for action in legal_actions(maze, node.pos):
            child = Node(transition(maze, node.pos, action), node, action, node.cost + 1)
            if child.pos in reached:
                continue
            if child.pos == goal:
                metrics["max_depth"] = max(metrics["max_depth"], child.cost)
                metrics["max_frontier"] = max(metrics["max_frontier"], len(frontier))
                metrics["max_memory"] = max(metrics["max_memory"], len(frontier) + len(reached))
                return child, metrics
            reached.add(child.pos)
            frontier.append(child)
            metrics["max_depth"] = max(metrics["max_depth"], child.cost)

        metrics["max_frontier"] = max(metrics["max_frontier"], len(frontier))
        metrics["max_memory"] = max(metrics["max_memory"], len(frontier) + len(reached))

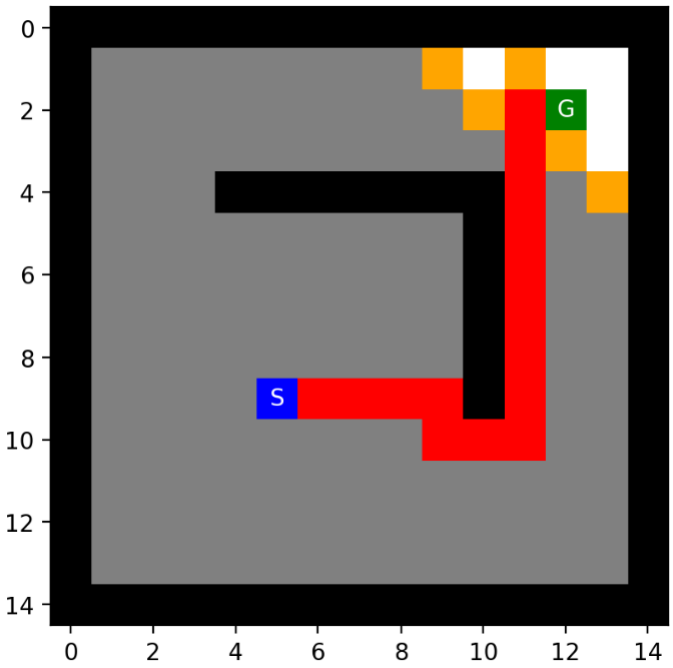
    return None, metrics
```

3.2. Kết quả BFS và DFS

Mỗi ô có định dạng: path cost / expanded / max depth / max memory / max frontier / thời gian (giây).

Mê cung	BFS: cost / expanded / depth / memory / frontier / s	DFS: cost / expanded / depth / memory / frontier / s
small	19 / 90 / 19 / 95 / 9 / 0.001	49 / 65 / 49 / 99 / 3 / 0.0
medium	68 / 267 / 68 / 272 / 9 / 0.001	74 / 76 / 74 / 149 / 3 / 0.0
large	210 / 618 / 210 / 625 / 8 / 0.006	210 / 529 / 222 / 445 / 4 / 0.004
L	16 / 146 / 16 / 156 / 17 / 0.001	102 / 109 / 102 / 205 / 4 / 0.001
loops	23 / 70 / 23 / 73 / 8 / 0.0	43 / 43 / 43 / 87 / 3 / 0.0

open	54 / 679 / 54 / 687 / 25 / 0.005	140 / 400000 / 268 / 537 / 4 / 3.234
empty	14 / 92 / 14 / 99 / 12 / 0.001	44 / 110 / 44 / 89 / 4 / 0.001
empty_2	14 / 92 / 14 / 99 / 11 / 0.001	50 / 360 / 58 / 117 / 4 / 0.003



Hình 4.1: So sánh đường đi của BFS (16 bước) và DFS (102 bước) trên L_maze.

3.3. Phân tích

- BFS cho path cost thấp hơn DFS ở small, medium, L, loops, open, empty và empty_2; ở large, hai output cùng cost 210. Điều này phù hợp với đặc tính tối ưu của BFS khi chi phí mỗi bước đồng nhất.
- Ở L_maze, BFS có cost 16 trong khi DFS có cost 102. DFS đi sâu theo thứ tự sinh trạng thái nên có thể tìm thấy một đường vòng dài trước khi khám phá đường ngắn hơn.
- Max frontier của DFS trong các output này nằm trong khoảng 3–4, còn BFS từ 8–25. Đây là minh họa cho việc DFS có thể giữ ít node chờ hơn, nhưng không có nghĩa DFS luôn nhanh hơn hoặc dùng ít bộ nhớ hơn theo mọi chỉ số.
- Open maze là trường hợp nổi bật: output DFS ghi expanded = 400000 và thời gian 3.234 giây, trong khi BFS có cost 54 và thời gian 0.005 giây. Kết quả này cho thấy hiệu năng DFS phụ thuộc mạnh vào thứ tự duyệt và cấu trúc mê cung.
- Các thời gian hiển thị 0.0 là giá trị được làm tròn trong output, không có nghĩa thuật toán chạy trong đúng 0 giây.

3.4. Tính đầy đủ, tối ưu và bộ nhớ

Tính đầy đủ là khả năng tìm thấy lời giải nếu lời giải tồn tại. BFS đầy đủ trong không gian hữu hạn với kiểm soát trạng thái phù hợp; DFS có thể gặp vấn đề khi đi sâu vô hạn hoặc lặp chu trình nếu không có cơ chế ngăn chặn. Tính tối ưu là khả năng tìm đường đi có chi phí thấp nhất. BFS tối ưu khi chi phí bước đồng nhất, còn DFS không bảo đảm tính chất này.

BÀI 4. DLS VÀ IDS

4.1. Depth-Limited Search (DLS)

DLS là DFS có giới hạn độ sâu L. Khi đạt giới hạn, thuật toán không mở rộng sâu hơn. Giới hạn giúp kiểm soát việc tìm kiếm nhưng nếu nhỏ hơn độ sâu lời giải, thuật toán có thể không tìm được đích.

Thông số DLS	Giá trị trong output
Depth limit	5
Frontier option	2
Reached squares	57
Wall time	1.98 ms

Output này không có các chỉ số path cost, expanded, max depth và max frontier; do đó các chỉ số này không được suy diễn từ output khác.

4.2. Iterative Deepening Search (IDS)

IDS chạy DLS nhiều lần với giới hạn độ sâu tăng dần. Thuật toán kết hợp ưu điểm tìm kiếm theo độ sâu của BFS với nhu cầu bộ nhớ thấp của DFS, nhưng phải duyệt lại các trạng thái ở những mức nông qua nhiều vòng.

```python
def depth_limited_search(maze, start, goal, limit, metrics):
    """DFS that does not descend below `limit` squares."""
    on_path = {start}

    def expand(node):
        metrics["expanded"] += 1
        for action in legal_actions(maze, node.pos):
            child_pos = transition(maze, node.pos, action)
            if child_pos in on_path:
                continue
            child = Node(child_pos, node, action, node.cost + 1)
            if child_pos == goal:
                return child
            if child.cost >= limit:
                continue
            on_path.add(child_pos)
            found = expand(child)
            on_path.discard(child_pos)
            if found is not None:
                return found
        return None

    return expand(Node(start, None, None, 0))
```

Mê cung	Số vòng	Path cost	Expanded	Thời gian (s)
small	19	19	923	0.00
medium	68	68	15429	0.08
L	14	Không tìm thấy lời giải trong output này	871232	5.34
loops	23	23	78683	0.42

IDS cần nhiều node mở rộng hơn một lần BFS vì các mức độ sâu nông được duyệt lại ở các vòng sau. Với L_maze, output IDS chính không ghi nhận lời giải trong cấu hình này; một output khác trong notebook khác ghi path length 16, nhưng không gộp hai kết quả vì chúng thuộc các lần chạy/cấu hình riêng biệt.

BÀI 5. TÌM KIẾM CÓ THÔNG TIN: GBFS, A* VÀ WEIGHTED A*

Trong phần này, ta sẽ phân tích sự khác biệt giữa các thuật toán tìm kiếm có thông tin (Informed Search) dựa trên kết quả chạy mô phỏng từ hai tập tin *02_Maze_Example.ipynb* và *04_Maze_BFS_vs_A_Star.ipynb*. Các thử nghiệm được quan sát trên không gian bài toán là mê cung *L_maze*.

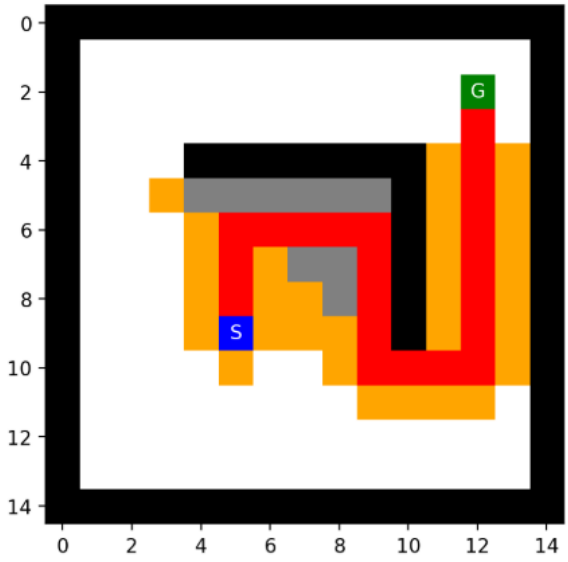
5.1. Greedy Best-First Search (GBFS)

Nguyên lý hoạt động: GBFS sử dụng hàm đánh giá $f(n) = h(n)$, tức là nó chỉ dựa vào hàm heuristic (khoảng cách ước lượng từ vị trí hiện tại đến đích) để quyết định bước đi tiếp theo. Thuật toán luôn ưu tiên chọn trạng thái có giá trị $h(n)$ nhỏ nhất.

Phân tích thực nghiệm: Kết quả từ file notebook cho thấy, trên mê cung *L_maze*, GBFS sử dụng Heuristic Manhattan tìm ra đường đi (Path length) là 22 và số nút phải mở rộng (Reached squares) là 61.

Đánh giá: GBFS mở rộng rất ít trạng thái do tính chất "tham lam" muốn lao nhanh về đích, khiến tốc độ duyệt cực nhanh. Tuy nhiên, nó bị đánh lừa bởi vật cản (bức tường chữ L) nên phải đi đường vòng, dẫn đến kết quả tìm được không phải là đường đi ngắn nhất (Path length = 22 > 16). Điều này minh chứng cho lý thuyết: GBFS nhanh nhưng **không bảo đảm tính tối ưu**.

```
Directions are checked at every step in random order.
CPU times: user 1.21 ms, sys: 119 µs, total: 1.33 ms
Wall time: 1.33 ms
Path length: 22
Reached squares: 61
```



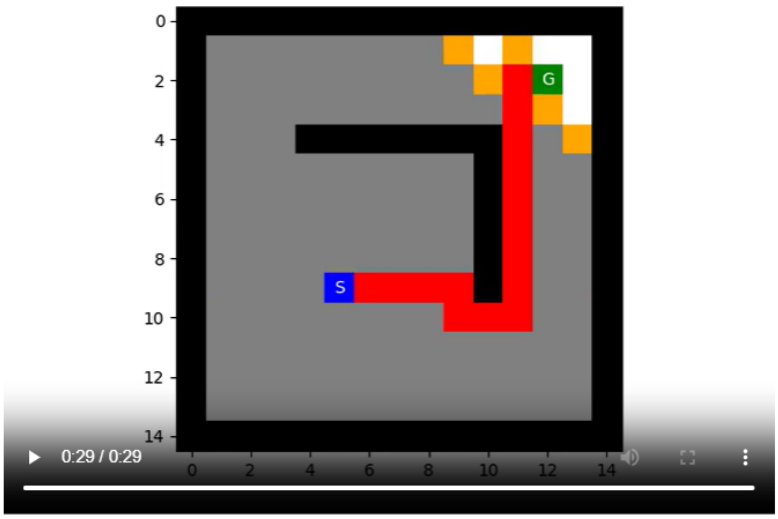
5.2. Thuật toán A*

Nguyên lý hoạt động: A* khắc phục nhược điểm của GBFS bằng cách kết hợp cả chi phí thực tế đã bỏ ra từ trạng thái ban đầu $g(n)$ và chi phí ước lượng đến đích $h(n)$. Hàm đánh giá là $f(n) = g(n) + h(n)$.

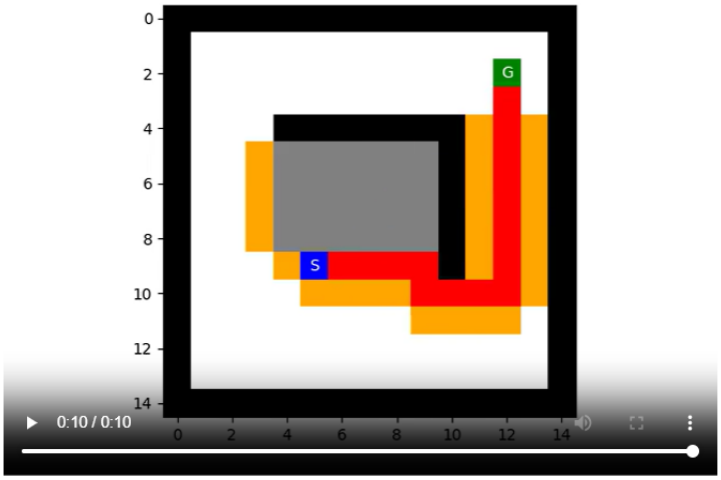
Phân tích thực nghiệm (So sánh trực tiếp BFS và A*):

- Theo kết quả trên *L_maze*, thuật toán tìm kiếm mù BFS tìm được đường đi ngắn nhất là 16 nhưng phải mở rộng không gian tìm kiếm ra đến 151 nút (Reached squares: 151).
- Trong khi đó, A* với Heuristic Manhattan cũng tìm ra đường đi tối ưu là 16 nhưng chỉ cần mở rộng 66 nút.
- Từ đó, đúng như nhận xét trong notebook: *BFS và A đều tìm đường ngắn nhất trong thiết lập này; A* có thể mở rộng ít trạng thái hơn đáng kể nếu heuristic phù hợp.

```
CPU times: user 4.11 ms, sys: 7.78 ms, total: 11.9 ms
Wall time: 11.4 ms
Path length: 16
Reached squares: 151
Action sequence: ['E', 'E', 'E', 'E', 'S', 'E', 'E', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'E']
```




```
CPU times: user 7.4 ms, sys: 0 ns, total: 7.4 ms
Wall time: 7.19 ms
Path length: 16
Reached squares: 66
Action sequence: ['E', 'E', 'E', 'E', 'S', 'E', 'E', 'E', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N']
```



Khảo sát hai Heuristic Euclidean và Manhattan đối với A*: Khi chạy A* trên L_maze bằng Heuristic Euclidean (đo khoảng cách đường chim bay $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$), số nút phải mở rộng là 87 nút. Trong khi dùng Heuristic Manhattan (đo khoảng cách theo lưới 4 hướng $h(n) = |x_n - x_g| + |y_n - y_g|$), số nút mở rộng giảm xuống chỉ còn 66 nút. Kết quả này chứng minh trên bài toán lưới vuông

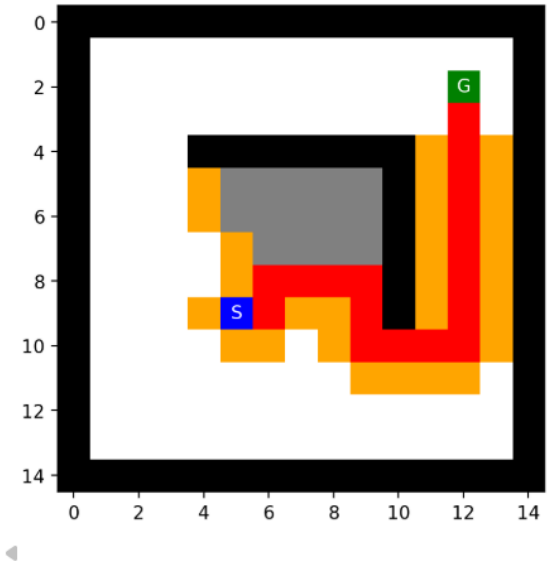
(không đi chéo), Manhattan ước lượng sát chi phí thực tế hơn Euclidean, do đó thu hẹp được vùng không gian tìm kiếm.

5.3. Thuật toán Weighted A*

Nguyên lý hoạt động: Thuật toán áp dụng trọng số w cho hàm heuristic: $f(n) = g(n) + w \cdot h(n)$. Việc này nhằm gia tăng ảnh hưởng của việc ước lượng về đích, giúp thuật toán định hướng mạnh hơn và giảm vùng tìm kiếm.

Phân tích thực nghiệm và sự đánh đổi: Khi thay đổi trọng số $W = 5$ cho Heuristic Manhattan trên *L_maze*, thuật toán chỉ cần mở rộng 59 nút (so với 66 nút của A* chuẩn, $W = 1$). Không gian tìm kiếm bị thu hẹp đáng kể, nghĩa là **tốc độ tìm kiếm tăng lên**. Tuy nhiên, sự đánh đổi là chi phí đường đi tìm được đã bị đẩy vòng xa hơn lên mức 18 (lớn hơn mức tối ưu là 16). Nhận xét này làm rõ lý thuyết: Weighted A* tăng định hướng heuristic để cải thiện tốc độ, nhưng sẽ đánh đổi **khả năng mất tính tối ưu khi** $w > 1$.

```
CPU times: user 1.85 ms, sys: 179 µs, total: 2.03 ms
Wall time: 2.07 ms
Path length: 18
Reached squares: 59
```



BÀI 6. ẢNH HƯỞNG CỦA HEURISTIC

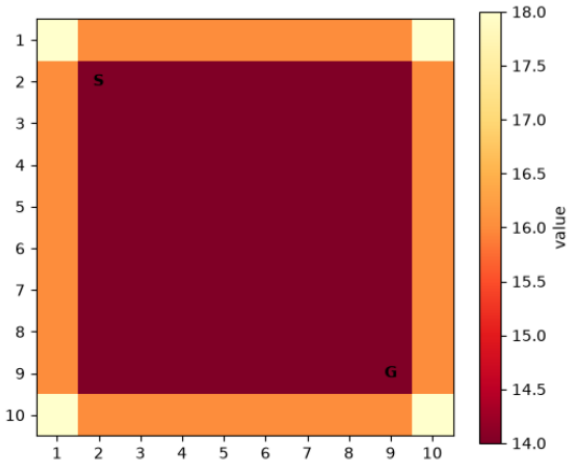
6.1. Quan sát bản đồ giá trị $h(n)$ và $f(n)$ trong mê cung rỗng

Bản đồ $h(n)$: Trong mê cung rỗng (empty maze), giá trị $h(n)$ (khoảng cách ước lượng đến đích) tạo thành các dải màu đồng tâm bao quanh nút đích. Do đặc tính khoảng cách Manhattan, các dải màu này có dạng hình thoi.

Bản đồ $f(n)$: Khi tính $f(n) = g(n) + h(n)$ bằng heuristic Manhattan, kết quả cho thấy một vùng "bằng phẳng" (plateau) rất lớn ở giữa mê cung nơi tất cả các trạng thái đều có cùng giá trị $f(n) = 16$. Mặc dù heuristic này là hoàn hảo ($h(n) = h^*(n)$), nhưng do có quá

nhiều hướng đi bị trùng điểm $f(n)$, thuật toán sẽ phải mở rộng một lượng lớn các nút (vùng màu đỏ) nếu không có chiến lược ưu tiên (tie-breaking) phù hợp, dẫn đến việc tìm kiếm kém hiệu quả. Để giải quyết, có thể dùng Weighted A^* với trọng số cực nhỏ (như $W = 1.1$) để tạo độ dốc nhẹ hướng thẳng về G.

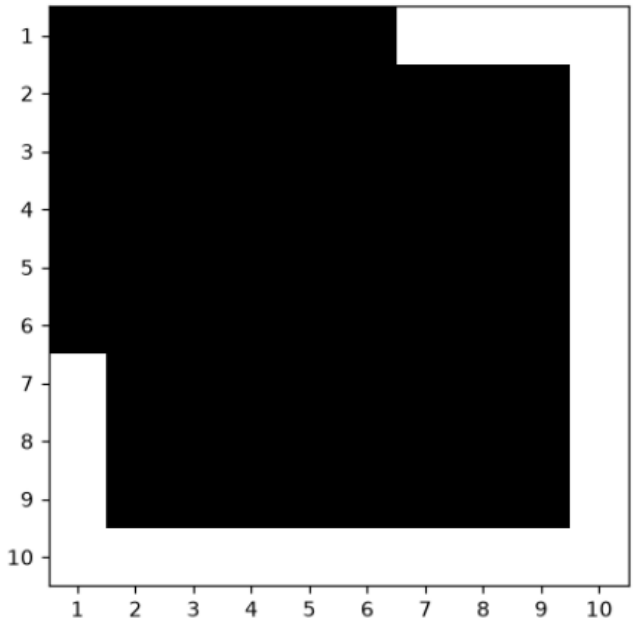
```
[[18 16 16 16 16 16 16 16 16 18]
 [16 14 14 14 14 14 14 14 14 16]
 [16 14 14 14 14 14 14 14 14 16]
 [16 14 14 14 14 14 14 14 14 16]
 [16 14 14 14 14 14 14 14 14 16]
 [16 14 14 14 14 14 14 14 14 16]
 [16 14 14 14 14 14 14 14 14 16]
 [16 14 14 14 14 14 14 14 14 16]
 [16 14 14 14 14 14 14 14 14 16]
 [18 16 16 16 16 16 16 16 16 18]]
```



6.2. So sánh Manhattan và Euclidean cho A^* trong lưới 4 hướng

- **Khoảng cách Manhattan** ($h(n) = |x_n - x_g| + |y_n - y_g|$) đếm số bước dịch chuyển chính xác theo hệ tọa độ lưới dọc và ngang. Trong lưới chỉ cho phép đi lên/xuống/trái/phải, nó phản ánh sát nhất chi phí thực tế tối thiểu.
- **Khoảng cách Euclidean** ($h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$) đo khoảng cách đường thẳng (đường chim bay). Trong không gian lưới 4 hướng, khoảng cách đường thẳng luôn ngắn hơn rất nhiều so với số bước đi thực tế, khiến hàm heuristic bị đánh giá quá thấp (underestimate).

Kết luận: File thực nghiệm chứng minh rằng hình học Euclidean và Manhattan "không tương thích tốt" trong bài toán này. Hàm $f(n)$ khi dùng Euclidean tạo ra một gradient ngược từ đích về điểm bắt đầu. Kết quả là A^* dùng Euclidean sẽ mở rộng gần như toàn bộ không gian của mê cung (tương đương với việc tìm kiếm mù BFS) để tìm đường, khiến thuật toán trở nên cực kỳ kém hiệu quả. Do đó, Manhattan luôn là lựa chọn ưu việt hơn trên lưới 4 hướng.



6.3. Đề xuất Heuristic mới và Điều kiện đảm bảo tính tối ưu

Điều kiện tiên quyết: Để thuật toán A^* luôn đảm bảo tính tối ưu (tìm được đường đi ngắn nhất), hàm Heuristic $h(n)$ được sử dụng bắt buộc phải mang tính chất **Chấp nhận được (Admissible)**. Nghĩa là, giá trị $h(n)$ ước lượng không bao giờ được phép lớn hơn chi phí di chuyển thực tế nhỏ nhất từ vị trí hiện tại đến đích ($h(n) \leq h^*(n)$).

Đề xuất Heuristic Chebyshev: Nếu cấu hình bài toán được mở rộng, cho phép tác tử di chuyển theo **8 hướng** (bổ sung 4 hướng đi chéo với cùng chi phí là 1), Heuristic Manhattan sẽ không còn admissible nữa vì nó ước lượng cao hơn chi phí đi chéo thực tế. Lúc này, ta đề xuất sử dụng khoảng cách Chebyshev với công thức: $h(n) = \max(|x_n - x_g|, |y_n - y_g|)$. Hàm này phản ánh chính xác số bước tối thiểu trong lưới 8 hướng, do đó nó admissible và giúp A^* giữ vững tính tối ưu.

Câu hỏi thảo luận

BÀI 7. CYCLE CHECKING

Cycle checking ngăn thuật toán lặp lại trạng thái trên đường đi hoặc trong toàn bộ quá trình tìm kiếm. Cơ chế này quan trọng với DFS vì mê cung có thể chứa chu trình và thứ tự sinh trạng thái có thể khiến thuật toán quay lại các vị trí đã đi qua.

Phương án	Kết quả ghi nhận	Thời gian
1. Kiểm tra trạng thái trên path	Không tìm thấy lời giải	34.1 ms
2. Kiểm tra path và tránh thêm trùng vào frontier	Không tìm thấy lời giải	43.9 ms
3. Kiểm tra path và điều	Path length = 54; Reached	51 ms

chỉnh frontier	squares = 0	

Output cho thấy thay đổi cách quản lý frontier có thể làm thay đổi kết quả tìm kiếm. Không nên kết luận chỉ từ thời gian rằng một phương án luôn tốt hơn: cần xét thêm việc có tìm được lời giải, path cost, số node mở rộng và tính đúng đắn của cách loại trạng thái trùng. Reached squares = 0 được giữ nguyên theo output; do chỉ số này không đi kèm định nghĩa chi tiết trong kết quả đã lưu, báo cáo không diễn giải nó thành số node mở rộng.

7.1. Thứ tự sinh trạng thái và chi phí quản lý

Thứ tự sinh trạng thái ảnh hưởng mạnh đến DFS vì thuật toán đi sâu theo nhánh được chọn trước. BFS ít nhạy hơn về độ dài lời giải khi mọi bước có chi phí bằng nhau, vì vẫn duyệt theo từng mức, nhưng thứ tự sinh có thể ảnh hưởng node nào được xét trước trong cùng một mức.

Cycle checking làm giảm việc xử lý trạng thái lặp, nhưng có thể tăng chi phí lưu trữ và tra cứu tập trạng thái đã gặp. Vì vậy, thiết kế phù hợp cần cân bằng giữa chi phí quản lý dữ liệu và lợi ích giảm tìm kiếm lặp.

BÀI 8. GOAL-BASED PLANNING AGENT VÀ NHIỀU MỤC TIÊU

Goal-based agent lựa chọn hành động dựa trên mục tiêu cần đạt. Agent biểu diễn trạng thái hiện tại, xét các hành động hợp lệ và sử dụng thuật toán tìm kiếm để xác định chuỗi hành động dẫn đến mục tiêu. Khác với phản ứng trực tiếp theo trạng thái hiện tại, agent có thể cân nhắc chuỗi hành động trong tương lai.

Với bài toán nhiều mục tiêu, việc tìm đường không chỉ cần xét vị trí hiện tại mà còn phải xác định điều kiện hoàn thành mục tiêu. Nếu mục tiêu là thăm tất cả các vị trí được yêu cầu, trạng thái tìm kiếm có thể cần bao gồm cả tập mục tiêu đã đạt được, không chỉ tọa độ hiện tại.

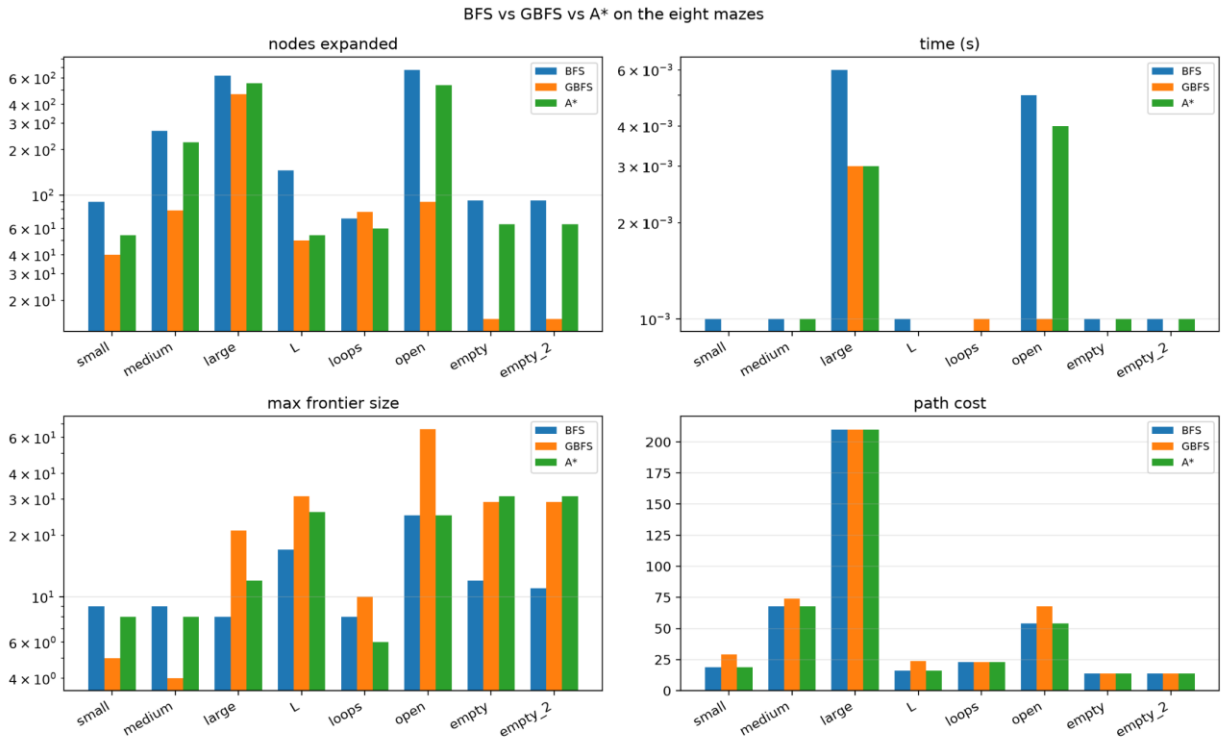
Thuật toán	Path cost	Expanded
BFS	27	71
A*	27	47
GBFS	27	77
DFS	125	143
IDS	27	874

Trong output nhiều mục tiêu, BFS, A* và IDS đều ghi path cost 27; A* mở rộng 47 node so với 71 của BFS, trong khi IDS mở rộng 874 node do lặp lại quá trình tìm kiếm theo giới hạn độ sâu. GBFS cũng có cost 27 trong trường hợp này, nhưng điều đó không bảo đảm GBFS luôn tối ưu. DFS có cost 125, cho thấy việc đi sâu theo một nhánh có thể tạo ra đường đi dài hơn đáng kể.

Kết quả trên phản ánh cấu hình nhiều mục tiêu cụ thể được thử nghiệm. Để so sánh các agent một cách công bằng, cần dùng cùng định nghĩa trạng thái, cùng điều kiện kết thúc và cùng quy tắc chi phí.

TỔNG HỢP KẾT QUẢ THỰC NGHIỆM

Thuật toán	Ưu điểm	Hạn chế / điều kiện
BFS	Tìm đường ngắn nhất khi chi phí mỗi bước bằng nhau; đầy đủ trong không gian hữu hạn với xử lý trạng thái phù hợp.	Frontier và bộ nhớ có thể tăng nhanh.
DFS	Có thể giữ frontier nhỏ; triển khai đơn giản.	Không bảo đảm tối ưu; nhạy với thứ tự sinh trạng thái; cần xử lý chu trình.
DLS	Kiểm soát độ sâu tìm kiếm.	Có thể bỏ lỡ lời giải nếu giới hạn quá nhỏ.
IDS	Tìm theo độ sâu tăng dần, thường tiết kiệm bộ nhớ hơn BFS.	Duyệt lại các trạng thái nông qua nhiều vòng.
GBFS	Có thể mở rộng ít node khi heuristic định hướng tốt.	Bỏ qua chi phí đã đi; không bảo đảm tối ưu.
A*	Kết hợp chi phí đã đi và ước lượng còn lại; tối ưu khi các điều kiện cần thiết được đáp ứng.	Hiệu quả phụ thuộc heuristic; có thể dùng nhiều bộ nhớ.
Weighted A*	Có thể giảm node mở rộng khi tăng trọng số heuristic.	Có thể tìm đường dài hơn tối ưu khi trọng số lớn hơn 1.



Nhìn tổng thể, không có thuật toán nào tốt nhất theo mọi tiêu chí. Nếu cần đường đi tối ưu với chi phí bước đồng nhất, BFS là mốc đối chiếu rõ ràng và A* với heuristic phù hợp có thể giảm lượng tìm kiếm. Nếu ưu tiên giảm số node khảo sát và chấp nhận khả năng đường đi dài hơn, GBFS hoặc Weighted A* có thể phù hợp hơn. DFS và IDS có lợi thế về chiến lược bộ nhớ nhưng cần cân nhắc thứ tự duyệt, chu trình và chi phí lặp lại.

BẢNG KẾT QUẢ SO SÁNH CÁC THUẬT TOÁN TÌM KIẾM

Bốn thuật toán BFS, DFS, GBFS và A* được chạy trên ba mê cung có đặc điểm khác nhau: small (mê cung nhỏ, có hành lang hẹp), medium (kích thước trung bình) và open (nhiều khoảng trống, nhiều chu trình). Heuristic của GBFS và A* là khoảng cách Manhattan; mỗi bước đi có chi phí 1 nên chi phí đường đi bằng số bước. Các chỉ số đo gồm: chi phí đường đi, số nút đã mở rộng, độ sâu cây tối đa, số nút tối đa trong bộ nhớ và kích thước frontier tối đa. Giá trị in đậm là giá trị tốt nhất (nhỏ nhất) trong từng cột.

Bảng 1. Kết quả trên mê cung Small

Thuật toán	Chi phí đường đi	Số nút đã mở rộng	Độ sâu cây tối đa	Số nút tối đa trong bộ nhớ	Kích thước frontier tối đa
BFS	**19**	90	**19**	95	9
DFS	49	65	49	99	**3**
GBFS	29	**40**	29	**49**	5
A*	**19**	54	**19**	66	8

Bảng 2. Kết quả trên mê cung Medium

Thuật toán	Chi phí đường đi	Số nút đã mở rộng	Độ sâu cây tối đa	Số nút tối đa trong bộ nhớ	Kích thước frontier tối đa
BFS	**68**	267	**68**	272	9
DFS	74	**76**	74	149	**3**
GBFS	74	79	74	**86**	4
A*	**68**	223	**68**	235	8

Bảng 3. Kết quả trên mê cung Open

Thuật toán	Chi phí đường đi	Số nút đã mở rộng	Độ sâu cây tối đa	Số nút tối đa trong bộ nhớ	Kích thước frontier tối đa
BFS	**54**	679	**54**	687	25

Thuật toán	Chi phí đường đi	Số nút đã mở rộng	Độ sâu cây tối đa	Số nút tối đa trong bộ nhớ	Kích thước frontier tối đa
DFS	140*	400000*	268	537	**4**
GBFS	68	**90**	68	**221**	66
A*	**54**	536	**54**	576	25

(*) DFS trên mê cung Open dừng khi chạm giới hạn 400000 nút; đường đi 140 bước là lời giải DFS tìm được, không phải kết quả hội tụ tự nhiên.

Bảng 4. Tổng hợp trên cả 8 mê cung (chi phí đường đi / số nút đã mở rộng)

Mê cung	BFS	DFS	GBFS	A*
Small	19 / 90	49 / 65	29 / 40	19 / 54
Medium	68 / 267	74 / 76	74 / 79	68 / 223
Large	210 / 618	210 / 529	210 / 467	210 / 550
L	16 / 146	102 / 109	24 / 50	16 / 54
Loops	23 / 70	43 / 43	23 / 77	23 / 60
Open	54 / 679	140* / 400000*	68 / 90	54 / 536
Empty	14 / 92	44 / 110	14 / 15	14 / 64
Empty (xoay)	14 / 92	50 / 360	14 / 15	14 / 64

(*) DFS chạm giới hạn 400000 nút trên mê cung Open.

Nhận xét từ bảng kết quả

- Tính tối ưu: BFS và A* luôn cho đường đi ngắn nhất trên cả 8 mê cung (ví dụ 19, 68 và 54 bước trên small, medium và open). GBFS chỉ tối ưu trên large, loops, empty và empty (xoay); trên small (29 so với 19), medium (74 so với 68), L (24 so với 16) và open (68 so với 54) nó cho đường đi dài hơn. DFS cho đường đi dài nhất trong hầu hết các mê cung, chẳng hạn 49 bước trên small và 140 bước trên open.

- Thời gian (số nút mở rộng): GBFS mở rộng ít nút nhất trên small (40), open (90) và empty (15) vì đi thẳng về phía đích. A* mở rộng ít hơn BFS ở hầu hết các mê cung (54 so với 90 trên small, 536 so với 679 trên open). DFS mở rộng ít nút trên medium (76) nhưng có thể rất tốn kém khi không bị giới hạn, như 400000 nút trên open.
- Bộ nhớ: BFS và A* cần nhiều bộ nhớ nhất vì phải lưu tập reached cùng frontier (687 và 576 nút trên open). DFS chỉ giữ frontier rất nhỏ (3 đến 4 nút) nhưng tổng số nút lưu lại vẫn có thể lớn khi đường đi dài.
- Vai trò của heuristic: Manhattan là heuristic khả chấp và rất sát với chi phí thực trong mê cung lưới, nên A* giữ được tính tối ưu mà vẫn mở rộng ít nút hơn BFS. GBFS bỏ qua g(n) nên nhanh nhưng dễ bị kẹt ở ngõ cụt rồi đi đường vòng.
- Cycle checking: DFS bắt buộc phải kiểm tra chu trình. Trên mê cung open nhiều khoảng trống, DFS sinh ra rất nhiều đường đi đơn khác nhau nên số nút mở rộng tăng vọt và chạm giới hạn.

NHẬN XÉT VÀ ĐÁNH GIÁ

Dựa trên kết quả thực nghiệm từ các cấu hình mê cung khác nhau, dưới đây là những phân tích và đánh giá chuyên sâu về hiệu năng của các thuật toán tìm kiếm theo các tiêu chí: tính đầy đủ, tính tối ưu, chi phí thời gian, bộ nhớ, hiệu quả của hàm Heuristic và vai trò của Cycle Checking.

1. Tính đầy đủ (Completeness) và Tính tối ưu (Optimality)

Tính đầy đủ: Đây là khả năng thuật toán chắc chắn tìm thấy lời giải nếu như lời giải thực sự tồn tại. Thuật toán BFS bảo đảm tính đầy đủ trong không gian hữu hạn khi được tích hợp kiểm soát trạng thái phù hợp. Các thuật toán có thông tin như A* cũng bảo đảm tính đầy đủ này. Ngược lại, DFS có thể gặp vấn đề nghiêm trọng về tính đầy đủ nếu đi sâu vô hạn hoặc lặp chu trình mà không có cơ chế ngăn chặn. Tương tự, DLS cũng không bảo đảm tính đầy đủ, thuật toán có thể bỏ lỡ lời giải nếu giới hạn độ sâu (limit) được thiết lập nhỏ hơn độ sâu của lời giải thực tế.

Tính tối ưu: Đây là khả năng tìm ra đường đi có chi phí (path cost) thấp nhất. Trong mô hình mê cung với chi phí mỗi bước đồng nhất, thuật toán BFS luôn thể hiện tính tối ưu. Thực nghiệm cho thấy BFS luôn tìm ra lời giải ngắn nhất trên các mê cung, ví dụ: small (19 bước), medium (68 bước), L_maze (16 bước). Thuật toán A* cũng bảo đảm tìm ra đường đi tối ưu nhờ kết hợp chi phí thực tế $g(n)$ và hàm heuristic $h(n)$ thoả mãn tính chấp nhận được (admissible). Ngược lại, DFS và GBFS không bảo đảm tính tối ưu. Điển hình ở L_maze, DFS sinh ra đường đi dài tới 102 bước do chiến lược đi sâu theo thứ tự sinh trạng thái và bám vào một nhánh vòng vèo. GBFS với bản chất "tham lam" cũng bị bẫy bởi vật cản chữ L, cho ra đường đi dài 22 bước thay vì 16 bước tối ưu.

2. Chi phí thời gian (Time) và Tiêu thụ bộ nhớ (Memory/Frontier)

Tiêu thụ bộ nhớ: Do duyệt không gian theo từng mức độ sâu, BFS tiêu tốn rất nhiều bộ nhớ. Tập biên (frontier) của BFS nở rộng nhanh theo cấp số nhân; max frontier của BFS thường dao động ở mức 8–25. Trong khi đó, DFS tiết kiệm bộ nhớ hơn hẳn khi chỉ duy trì max frontier ở mức 3–4 nhờ chiến lược duyệt theo chiều sâu. Thuật toán IDS dung hòa được nhu cầu bộ nhớ thấp của DFS và tính rà soát theo mức của BFS, nhưng đánh đổi bằng việc duyệt lặp lại các trạng thái nông qua nhiều vòng.

Thời gian thực thi: Tốc độ của thuật toán tìm kiếm mù phụ thuộc mạnh vào cấu trúc mê cung. Ở mê cung dạng mở (open maze), DFS mở rộng tới 400.000 node và mất hơn 3.2 giây để hoàn thành, trong khi BFS chỉ mất 0.005 giây. Đối với nhóm tìm kiếm có thông tin, GBFS có tốc độ cực nhanh do ưu tiên mục tiêu cục bộ (chỉ mở rộng 61 node trên L_maze), nhưng dễ bị kẹt ở ngõ cụt. A* là sự cân bằng hoàn hảo nhất: nhờ định hướng từ heuristic, nó chỉ cần mở rộng 66 node để tìm ra đường đi tối ưu (so với 151 node của BFS), giúp tối ưu hóa cả thời gian và bộ nhớ.

3. Đánh giá ảnh hưởng của hàm Heuristic

- Hàm heuristic $h(n)$ tốt có khả năng làm giảm đáng kể vùng trạng thái phải khảo sát.
- Trên môi trường lưới vuông chỉ cho phép đi 4 hướng cơ bản, khoảng cách Manhattan (L1) ước lượng rất sát với số bước di chuyển thực tế tối thiểu. Do đó, Manhattan thoả mãn tính admissible và giúp A* rà soát cực kỳ hiệu quả.
- Ngược lại, khoảng cách Euclidean (L2) đo theo đường chim bay luôn ngắn hơn rất nhiều so với số bước đi thực tế trên lưới. Heuristic Euclidean đánh giá thấp chi phí (underestimate) quá mức, khiến gradient hướng đích bị nhạt dần. Kết quả là A* phải rà soát lan rộng ra đến 87 node (gần tương đương với tìm kiếm mù).
- Khi sử dụng thuật toán Weighted A* với trọng số $w = 5$, tính định hướng được khuếch đại, giúp giảm số node rà soát xuống chỉ còn 59. Tuy nhiên, chi phí đường đi tăng lên 18, minh chứng rõ ràng cho việc Weighted A* đánh đổi tính tối ưu để lấy tốc độ duyệt.

4. Vai trò của Cycle Checking (Kiểm tra chu trình)

Trong các môi trường phức tạp có chứa chu trình, Cycle checking là cơ chế tối quan trọng. Nó đặc biệt ảnh hưởng lớn đến DFS, vì DFS có thể quay lại các vị trí đã đi qua và rơi vào vòng lặp vô tận nếu không có cơ chế chặn.

Việc áp dụng Cycle checking đồng nghĩa với việc tạo ra chi phí quản lý dữ liệu. Thuật toán phải tốn thêm thời gian (như trong thực nghiệm là 34.1 ms đến 51 ms tùy phương án) và bộ nhớ để lưu trữ, cập nhật, tra cứu tập trạng thái đã thăm (visited/reached set).

Tuy nhiên, đây là sự đánh đổi hoàn toàn xứng đáng và bắt buộc: chi phí overhead này giúp thuật toán hội tụ (kết thúc được), tránh bùng nổ không gian do lặp vô ích. Dù vậy,

Cycle checking chỉ đóng vai trò ngăn vòng lặp, nó không thể khắc phục được nhược điểm cốt lõi của DFS nên không thể giúp DFS trở thành thuật toán tìm đường tối ưu.

B. CÂU HỎI THẢO LUẬN

Câu 6: Khi nào BFS dùng nhiều bộ nhớ hơn A*? Vì sao heuristic tốt làm giảm vùng trạng thái phải khảo sát?

BFS dùng nhiều bộ nhớ hơn A* khi:

- Thuật toán BFS duyệt không gian trạng thái theo từng mức độ sâu đồng tâm và không có định hướng mục tiêu. Do đó, tập biên (*frontier*) của BFS nở rộng rất nhanh theo cấp số nhân $O(b^d)$ (với b là hệ số nhánh, d là độ sâu nghiệm). Toàn bộ các nút thuộc mức hiện tại và mức kế tiếp đều phải lưu đồng thời trong RAM.
- Ngược lại, A* sử dụng hàm đánh giá $f(n) = g(n) + h(n)$ để tập trung duyệt về hướng có triển vọng nhất. Khi bài toán có không gian mở hoặc khoảng cách tới đích tương đối xa, A* chỉ mở rộng một dải trạng thái hẹp bám sát đường đi tiềm năng. Vì vậy, trong hầu hết các cấu hình bài toán có mục tiêu cụ thể, BFS sẽ tiêu tốn bộ nhớ lớn hơn A* rất nhiều lần.

Heuristic tốt làm giảm vùng trạng thái phải khảo sát vì:

- Hàm heuristic $h(n)$ ước lượng khoảng cách còn lại từ trạng thái hiện tại về đích. Một hàm heuristic được xem là tốt khi nó thoả mãn tính chấp nhận được (*admissible*) và ước lượng sát với chi phí thực tế $h^*(n)$.
- Khi $h(n)$ sát thực tế, tổng chi phí dự kiến $f(n)$ của các nhánh đi chệch hướng sẽ tăng vọt rất nhanh, khiến hàng đợi ưu tiên đẩy chúng xuống đáy. Thuật toán sẽ tập trung tối đa việc mở rộng các nhánh dẫn trực diện về đích, loại bỏ việc rà soát các nhánh rỗng/vô ích, từ đó làm co hẹp đáng kể vùng trạng thái khảo sát (*reached/explored squares*).

Câu 7. Vì sao DFS có thể cho đường đi dài hoặc gặp vấn đề trong môi trường có chu trình?

DFS ưu tiên mở rộng trạng thái sâu nhất trong frontier bằng cấu trúc ngăn xếp LIFO, nên kết quả phụ thuộc vào nhánh được chọn và thứ tự sinh các trạng thái con. Do không ưu tiên đường đi có chi phí thấp nhất, DFS có thể tìm được lời giải nhưng đường đi dài hơn đường tối ưu. Trong môi trường có chu trình, nếu không kiểm tra trạng thái lặp, thuật toán có thể quay lại các trạng thái đã gặp nhiều lần, làm tăng đáng kể thời gian tìm kiếm hoặc khiến quá trình tìm kiếm không kết thúc. Vì vậy, cần áp dụng cycle checking hoặc tập trạng thái đã thăm để hạn chế việc mở rộng lặp. Dù vậy, cycle checking chỉ giúp kiểm soát việc lặp trạng thái, không làm cho DFS trở thành thuật toán tối ưu

Câu 8. Vì sao thứ tự sinh trạng thái con ảnh hưởng mạnh đến DFS nhưng ít ảnh hưởng hơn đến BFS?

Thứ tự sinh trạng thái con ảnh hưởng mạnh đến DFS vì thuật toán sử dụng ngăn xếp LIFO và ưu tiên đi sâu theo trạng thái được lấy ra trước. Việc thay đổi thứ tự sinh có thể khiến DFS lựa chọn nhánh khác, dẫn đến sự khác biệt lớn về đường đi tìm được, số trạng thái mở rộng và thời gian thực hiện. Ngược lại, BFS sử dụng hàng đợi FIFO và luôn xét các trạng thái theo thứ tự độ sâu tăng dần. Vì vậy, khi mọi bước có cùng chi phí, thứ tự sinh trạng thái thường không làm thay đổi độ dài của lời giải đầu tiên, nhưng vẫn có thể ảnh hưởng đến đường đi cụ thể được chọn khi có nhiều lời giải ngắn nhất, cũng như số trạng thái mở rộng và thời gian chạy.

Câu 9: So sánh GBFS và A*: thuật toán nào sử dụng $g(n)$, $h(n)$, $f(n)$? Ảnh hưởng đến tính tối ưu như thế nào?

Việc sử dụng các hàm chi phí:

- **GBFS (Greedy Best-First Search):** Chỉ sử dụng ước lượng khoảng cách tới đích $h(n)$ làm hàm đánh giá: $f(n) = h(n)$. GBFS hoàn toàn bỏ qua chi phí thực tế đã tích luỹ từ điểm xuất phát $g(n)$.
- **Thuật toán A*:** Kết hợp đồng thời cả chi phí thực tế đã đi $g(n)$ và chi phí ước lượng còn lại $h(n)$ thông qua hàm tổng quát: $f(n) = g(n) + h(n)$.

Ảnh hưởng đến tính tối ưu:

- **GBFS không bảo đảm tính tối ưu:** Do tính "tham lam" chỉ chọn bước đi có vẻ gần đích nhất tại thời điểm cục bộ, GBFS rất dễ bị bẫy bởi các vật cản (như bức tường chữ L trên mê cung L_maze). Thuật toán buộc phải đi vòng khi gặp ngõ cụt, dẫn đến chiều dài đường đi tìm được dài hơn đáng kể so với mức tối ưu (thực nghiệm: Path length = 22 thay vì 16).
- **A* bảo đảm tính tối ưu:** Nhờ có thành phần $g(n)$ kìm hãm sự "tham lam" quá mức, cùng với điều kiện hàm $h(n)$ chấp nhận được (*admissible*), A* bảo đảm luôn tìm ra đường đi ngắn nhất thực tế (thực nghiệm: Path length = 16).

Câu 10: Manhattan và Euclidean khác nhau ra sao trong lưới chỉ cho phép đi lên/xuống/trái/phải?

Bản chất tính toán:

- **Khoảng cách Manhattan (L_1):** Tính theo công thức $h(n) = |x_n - x_g| + |y_n - y_g|$. Nó đo chính xác số bước di chuyển tối thiểu theo các cạnh lưới vuông.
- **Khoảng cách Euclidean (L_2):** Tính theo khoảng cách đường chim bay: $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$.

Sự khác biệt trong lưới 4 hướng:

- Trong lưới chỉ cho phép đi 4 hướng cơ bản (không cho phép đi chéo), tác tử buộc phải di chuyển theo các cạnh vuông góc. Do đó, khoảng cách đường thẳng Euclidean luôn ngắn hơn đáng kể so với chi phí di chuyển tối thiểu thực tế, tức là $h_{Euclidean}(n) \leq h_{Manhattan}(n) \leq h^*(n)$.
- Hệ quả là heuristic Euclidean đánh giá thấp chi phí quá mức (*underestimate* quá nhiều). Khi chạy A*, heuristic Euclidean định hướng kém hơn, làm phân rã gradient hướng đích và khiến thuật toán phải mở rộng vùng tìm kiếm lớn hơn rất nhiều (thực nghiệm: 87 nút so với 66 nút của Manhattan). Vì vậy, Manhattan luôn là heuristic phù hợp và vượt trội hơn trên lưới 4 hướng.

Câu 11: Weighted A* thay đổi điều gì khi tăng trọng số heuristic?

Weighted A* sử dụng hàm đánh giá: $f(n) = g(n) + w \cdot h(n)$ với trọng số $w > 0$.

Khi tăng trọng số w (đặc biệt khi $w > 1$):

- **Gia tăng tính định hướng và tốc độ:** Trọng số w phóng đại tầm ảnh hưởng của heuristic $h(n)$, làm thuật toán nghiêng dần về bản chất "tham lam" giống GBFS. Thuật toán tập trung lao thẳng về phía mục tiêu, nhờ đó kích thước tập biên co lại đáng kể và số nút phải mở rộng giảm đi rõ rệt (thực nghiệm: $w = 5$ chỉ cần mở rộng 59 nút so với 66 nút của A* chuẩn). Tốc độ tìm kiếm được đẩy lên cao hơn.
- **Đánh đổi tính tối ưu:** Khi $w > 1$, hàm $w \cdot h(n)$ có thể vượt quá chi phí thực tế $h^*(n)$ (vi phạm tính *admissible*). Khi đó, Weighted A* không còn đảm bảo tìm ra đường đi ngắn nhất nữa, chiều dài đường đi tìm được có thể bị dài hơn mức tối ưu (thực nghiệm: chi phí đường đi tăng lên 18 thay vì 16).

Câu 12. Cycle checking giảm lặp trạng thái nhưng làm tăng chi phí quản lý dữ liệu như thế nào?

Cycle checking giúp giảm việc lặp trạng thái bằng cách kiểm tra xem trạng thái mới đã được gặp hoặc mở rộng trước đó hay chưa. Nếu trạng thái đã được ghi nhận, thuật toán có thể bỏ qua việc xử lý trùng lặp, nhờ đó giảm số lần mở rộng và tránh lặp vô ích trong môi trường có chu trình. Tuy nhiên, cơ chế này làm phát sinh chi phí lưu trữ tập trạng thái đã thăm, tra cứu thành viên và cập nhật dữ liệu. Chi phí tra cứu phụ thuộc vào cấu trúc dữ liệu được sử dụng; chẳng hạn, tập dựa trên bảng băm thường hỗ trợ tra cứu trung bình nhanh hơn danh sách tuyến tính. Nhìn chung, cycle checking là sự đánh đổi giữa bộ nhớ và thời gian quản lý với lợi ích giảm công việc tìm kiếm lặp lại.

C. KẾT LUẬN

Bài thực hành cho thấy việc lựa chọn chiến lược tìm kiếm cần dựa trên yêu cầu về tính tối ưu, thời gian, bộ nhớ và đặc điểm của không gian trạng thái. BFS cho đường đi ngắn nhất trong mô hình chi phí bước đồng nhất nhưng có thể cần frontier lớn. DFS có thể tiết kiệm frontier nhưng đường đi phụ thuộc thứ tự duyệt và cần cơ chế kiểm tra chu trình. DLS

và IDS kiểm soát độ sâu theo những cách khác nhau, trong đó IDS đánh đổi việc duyệt lặp để không phải chọn trước một giới hạn cố định.

Trong tìm kiếm có thông tin, GBFS có thể mở rộng ít node nhưng không cân nhắc chi phí đã đi. A* kết hợp g(n) và h(n), còn Weighted A* tăng ảnh hưởng của heuristic để có thể giảm số node mở rộng với nguy cơ mất tối ưu. Các kết quả Manhattan và Euclidean cho thấy lựa chọn heuristic ảnh hưởng trực tiếp đến lượng tìm kiếm. Những số liệu thực nghiệm cụ thể là căn cứ chính cho các nhận xét, trong khi các kết luận lý thuyết cần được hiểu trong điều kiện áp dụng tương ứng.

BẢNG PHÂN CÔNG:

MSSV	Họ Tên	Phân công
3124720026	Huỳnh Tấn Phi Hùng	Viết báo cáo cho bài 5, 6 và trả lời các câu hỏi 6, 9, 10, 11
3124720072	Đỗ Xuân Triệu	Lập trình bài ex01, ex02, ex 03
3124720050	Lê Tấn Phát	Viết báo cáo cho bài 1, 2, 3, 4, 7, 8 và trả lời các câu hỏi 7, 8, 12